

Skill - 11:

```
sahasrasagi@Sahasra: ~/OS_L  ×  +  ▾
GNU nano 8.7.1 skill_11.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_HISTORY 10
#define MAX_COMMAND 100
#define MAX_PIPELINE 10

char history[MAX_HISTORY][MAX_COMMAND];
int history_count = 0;

char pipeline[MAX_PIPELINE][MAX_COMMAND];
int pipeline_count = 0;

void add_history(char *command)
{
    if (history_count < MAX_HISTORY)
    {
        strcpy(history[history_count], command);
        history_count++;
    }
    else
    {
        for (int i = 0; i < MAX_HISTORY - 1; i++)
        {
            strcpy(history[i], history[i + 1]);
        }
        strcpy(history[MAX_HISTORY - 1], command);
    }
}

void display_history()
{
    printf("\nCommand History:\n");
    for (int i = 0; i < history_count; i++)
    {
        printf("%d %s\n", i + 1, history[i]);
    }
}

void create_pipeline(char *input)
{
    char temp[MAX_COMMAND];
    strcpy(temp, input);

    char *token = strtok(temp, "|");
```

```
    char *token = strtok(temp, "|");

    pipeline_count = 0;

    while (token != NULL && pipeline_count < MAX_PIPELINE)
    {
        while (*token == ' ')
            token++;

        strcpy(pipeline[pipeline_count], token);
        pipeline_count++;

        token = strtok(NULL, "|");
    }
}
```

```
sahasrasagi@Sahasra: ~/OS_L  ×  +  ∨
GNU nano 8.7.1 skill11
void display_pipeline()
{
    printf("\nPipeline Structure:\n");
    for (int i = 0; i < pipeline_count; i++)
    {
        printf("Process %d: %s\n", i + 1, pipeline[i]);
        if (i < pipeline_count - 1)
        {
            printf("      |\n");
            printf("      v\n");
        }
    }
}

int main()
{
    char command[MAX_COMMAND];
    char pipeline_input[MAX_COMMAND];

    printf("==== Skill 11 =====\n");
    printf("\nEnter 3 commands:\n");
    for (int i = 0; i < 3; i++)
    {
        printf("Command %d: ", i + 1);

        fgets(command, sizeof(command), stdin);
        command[strcspn(command, "\n")] = '\0';

        add_history(command);
    }

    display_history();

    printf("\nEnter a pipeline command:\n");
    printf("Pipeline: ");

    fgets(pipeline_input, sizeof(pipeline_input), stdin);
    pipeline_input[strcspn(pipeline_input, "\n")] = '\0';

    create_pipeline(pipeline_input);
    display_pipeline();

    return 0;
}

^G Help  ^O Write Out  ^F Where Is  ^K Cut  ^T Execute  ^C Location
```

```
sahasrasagi@Sahasra:~/OS_Lab/skill$ gcc skill_11.c -o skill_11
sahasrasagi@Sahasra:~/OS_Lab/skill$ ./skill_11
==== Skill 11 =====

Enter 3 commands:
Command 1: pwd
Command 2: ls
Command 3: date

Command History:
1  pwd
2  ls
3  date

Enter a pipeline command:
Pipeline: ls

Pipeline Structure:
Process 1: ls
sahasrasagi@Sahasra:~/OS_Lab/skill$ |
```

Skill-12:

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_COMMAND 100
#define MAX_ARGS 20

void execute_command(char *command)
{
    char *args[MAX_ARGS];
    int count = 0;

    char *token = strtok(command, " ");

    while (token != NULL && count < MAX_ARGS - 1)
    {
        args[count++] = token;
        token = strtok(NULL, " ");
    }

    args[count] = NULL;

    execvp(args[0], args);

    perror("execvp");
    exit(1);
}

void execute_pipeline(char *input)
{
    char *commands[10];
    int command_count = 0;

    char *token = strtok(input, "|");

    while (token != NULL && command_count < 10)
    {
        commands[command_count++] = token;
        token = strtok(NULL, "|");
    }

    int pipes[9][2];
    pid_t pids[10];

    for (int i = 0; i < command_count - 1; i++)
    {
        if (pipe(pipes[i]) == -1)
        {
            perror("pipe");
            exit(1);
        }
    }

    for (int i = 0; i < command_count; i++)
    {
        pids[i] = fork();

        if (pids[i] == -1)
        {
            perror("fork");
            exit(1);
        }
    }
}
```

```

for (int i = 0; i < command_count; i++)
{
    pids[i] = fork();

    if (pids[i] == -1)
    {
        perror("fork");
        exit(1);
    }

    if (pids[i] == 0)
    {
        if (i > 0)
        {
            dup2(pipes[i - 1][0], STDIN_FILENO);
        }

        if (i < command_count - 1)
        {
            dup2(pipes[i][1], STDOUT_FILENO);
        }

        for (int j = 0; j < command_count - 1; j++)
        {
            close(pipes[j][0]);
            close(pipes[j][1]);
        }

        while (*commands[i] == ' ')
            commands[i]++;

        execute_command(commands[i]);
    }
}

for (int i = 0; i < command_count - 1; i++)
{
    close(pipes[i][0]);
    close(pipes[i][1]);
}

for (int i = 0; i < command_count; i++)
{
    waitpid(pids[i], NULL, 0);
}
}

int main()
{
    char input[MAX_COMMAND];

    printf("==== Skill 12 =====\n");
    printf("Enter pipeline: ");

    fgets(input, sizeof(input), stdin);
    input[strcspn(input, "\n")] = '\0';

    execute_pipeline(input);

    return 0;
}

```

```

sahasrasagi@Sahasra:~/OS_Lab/skill$ gcc skill_12.c -o skill_12
sahasrasagi@Sahasra:~/OS_Lab/skill$ ./skill_12
==== Skill 12 =====
Enter pipeline: ls
a.out      program1.c  program2.c  shellforge.c  skill_10.c  skill_11.c
program1   program2     shellforge  skill_10      skill_11    skill_12
sahasrasagi@Sahasra:~/OS_Lab/skill$ |

```

Skill- 13:

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

#define MAX_INPUT 200
#define MAX_ARGS 20

void execute_command(char *input)
{
    char *args[MAX_ARGS];
    int count = 0;
    int fd;

    char *redirect = strchr(input, '<');

    if (redirect != NULL)
    {
        *redirect = '\0';
        redirect++;

        while (*redirect == ' ')
            redirect++;

        char *filename = strtok(redirect, " ");

        if (filename == NULL)
        {
            printf("Missing input file\n");
            return;
        }

        fd = open(filename, O_RDONLY);

        if (fd == -1)
        {
            perror("open");
            return;
        }
    }

    char *token = strtok(input, " ");

    while (token != NULL && count < MAX_ARGS - 1)
    {
        args[count++] = token;
        token = strtok(NULL, " ");
    }

    args[count] = NULL;

    pid_t pid = fork();

    if (pid == -1)
    {
        perror("fork");
        return;
    }
}
```

```

    }

    if (pid == 0)
    {
        if (redirect != NULL)
        {
            dup2(fd, STDIN_FILENO);
            close(fd);
        }

        execvp(args[0], args);

        perror("execvp");
        exit(1);
    }

    waitpid(pid, NULL, 0);

    if (redirect != NULL)
        close(fd);
}

int main()
{
    char input[MAX_INPUT];

    printf("==== Skill 13 =====\n");
    printf("Enter command: ");

    fgets(input, sizeof(input), stdin);
    input[strcspn(input, "\n")] = '\0';

    execute_command(input);

    return 0;
}

```

```

program1  program2  shectforge  skill_10  skill_11  skill_12
sahasrasagi@Sahasra:~/OS_Lab/skill$ nano skill_13.c
sahasrasagi@Sahasra:~/OS_Lab/skill$ gcc skill_13.c -o skill_13
sahasrasagi@Sahasra:~/OS_Lab/skill$ ./skill_13
==== Skill 13 =====
Enter command: echo "Hello from input file" >input.txt
"Hello from input file" >input.txt

```